

make

Lecture 16

Dr. Colin Rundel

make

- build tool for the creation of software / libraries / documents by specifying dependencies
 - Almost any process that has files as input and outputs can be automated via make
- Originally created by Stuart Feldman in 1976 at Bell Labs
- Almost universally available (all flavors of unix / linux / MacOS / Windows via RTools)
- Dependencies are specified using a text-based `Makefile` with a simple syntax

Makefile

A **Makefile** provides a list of target files along, their dependencies, and the steps necessary to generate each of the targets from the dependencies.

```
1 target1: depend1 depend2 depend3 ...
2     step1
3     step2
4     step3
5     ...
6
7 depend1: depend4 depend5
8     step1
9     step2
10    ...
```

In the above example **target*** and **depend*** are all just files (given by a relative or absolute path).

Makefile (basic example)

```
1 paper.html: paper.Rmd fig1/fig.png fig2/fig.png
2     Rscript -e "rmarkdown::render('paper.Rmd')"
3
4 fig1/fig.png: fig1/fig.R
5     Rscript fig1/fig.R
6
7 fig2/fig.png: fig2/fig.R
8     Rscript fig2/fig.R
```

Smart Building

Because the `Makefile` specifies the dependency structure `make` knows when a file has changed (by examining the file's modification timestamp) and only runs the steps that depend on the file(s) that have changed.

- After running `make` the first time, I edit `paper.Rmd`, what steps run if I run `make` again?
- What about editing `fig1/fig.R`?

Variables

Like R or other language we can define variables

```
1 R_OPTS=--no-save --no-restore --no-site-file --no-init-file .
2
3 fig1/fig.png: fig1/fig.R
4     cd fig1;Rscript $(R_OPTS) fig.R
```

Special Targets

By default if you run `make` without arguments it will attempt to build the *first* target in the `Makefile` (whose name does not start with a `.`). By convention we often include an `all` target which explicitly specifies how to build everything within the project.

`all` is an example of what is called a phony target - because there is no file named `all` in the directory. Other common phony targets:

- `clean` - remove any files created by the Makefile, restores to the original state
- `install` - for software packages, installs the compiled programs / libraries / header files

Optionally, we specify all phony targets by including a line with `.PHONY` as the target and the phony targets as dependencies, i.e.:

```
1 .PHONY: all clean install
```

Builtin / Automatic Variables

- `$@` - the file name of the target
- `$<` - the name of the first dependency
- `$^` - the names of all dependencies
- `$(@D)` - the directory part of the target
- `$(@F)` - the file part of the target
- `$(<D)` - the directory part of the first dependency
- `$(<F)` - the file part of the first dependency

Pattern Rules

Often we want to build several files in the same way, in these cases we can use `%` as a special wildcard character to match both targets and dependencies.

So we can go from

```
1 fig1/fig.png: fig1/fig.R
2     cd fig1;Rscript fig.R
3
4 fig2/fig.png: fig2/fig.R
5     cd fig2;Rscript fig.R
```

to

```
1 fig%/fig.png: fig%/fig.R
2     cd $(<D);Rscript $(<F)
```

Makefile (fancier example)

```
1 all: paper.html
2
3 paper.html: paper.Rmd fig1/fig.png fig2/fig.png
4     Rscript -e "library(rmarkdown);render('paper.Rmd')"
5
6 Fig%/fig.png: Fig%/fig.R
7     cd $(<D);Rscript $(<F)
8
9 clean:
10     rm -f paper.html
11     rm -f Fig*/*.png
12
13 .PHONY: all clean
```

Live Demo

HW4 Makefile

HW4 Makefile

```
1 all: hw4.html
2
3 hw4.html: hw4.qmd data/lq.rds data/dennys.rds
4     quarto render hw4.qmd
5
6 data/lq.rds: parse_lq.R data/lq/*.html
7     Rscript parse_lq.R
8
9 data/lq/*.html: get_lq.R
10    Rscript get_lq.R
11
12 data/dennys.rds: parse_dennys.R data/dennys/*.html
13    Rscript parse_dennys.R
14
15 data/dennys/*.html: get_dennys.R
16    Rscript get_dennys.R
17
18 clean:
19     rm -f hw4.html
20     rm -rf data/
```



Why targets?

`make` is great, but has some limitations for data analysis pipelines:

- File-based dependencies only (what about R objects in memory?)
- Limited insight into what changed and why
- Limited support for parallel execution
- Syntax is shell based

The `targets` package provides a modern R-native alternative:

- Define pipelines in R code
- Track both files and R objects
- Automatic parallel execution
- Better debugging and visualization
- Integration with R ecosystem (Quarto, R Markdown, etc.)

Getting started with targets

```
1 # Create a new targets project
2 use_targets()
3
4 # Explore your pipeline
5 tar_visnetwork()
6 tar_manifest()
7
8 # Run the pipeline
9 tar_make()
10
11 # Check what needs updating
12 tar_outdated()
13
14 # Access a target from storage
15 tar_read()
16
17 # To reset / clean-up
18 tar_invalidate()
19 tar_prune()
20 tar_destroy()
```

Basic targets workflow

A `targets` pipeline is defined in a `_targets.R` file:

```
1 library(targets)
2
3 tar_option_set(packages = c("tibble", "dplyr"))
4
5 list(
6   tar_target(file, "data.csv", format = "file"),
7   tar_target(raw_data, read.csv(file)),
8   tar_target(clean_data, raw_data |> filter(!is.na(value))),
9   tar_target(summary, summarize(clean_data, mean = mean(value)))
10 )
```

Run the pipeline:

```
1 tar_visnetwork()
2 tar_make()
3 tar_read(summary)
```


HW4 pipeline with targets

```
1 # _targets.R
2 library(targets)
3
4 list(
5   # Get La Quinta data
6   tar_target(lq_html_files, {
7     source("get_lq.R")
8     list.files("data/lq", pattern = "*.html", full.names = TRUE)
9   }),
10
11   tar_target(lq_data, {
12     source("parse_lq.R")
13     readRDS("data/lq.rds")
14   }, depend_on = lq_html_files),
15
16   # Get Denny's data
17   tar_target(dennys_html_files, {
18     source("get_dennys.R")
19     list.files("data/dennys", pattern = "*.html", full.names = TRUE)
20   }),
```